

Assignment: 3

Unit: 2

Name: Kavshal Kumar Raj

PRN: 240205241023

Branch: B.Tech CSE AI & ML

Semester: IV

Subject: Deep Learning

① What is Gradient Descent and why is it used in deep learning?

→ Gradient Descent is an optimization algorithm used to minimize a model's loss (error) function by iteratively adjusting the model's parameters in the direction of the negative gradient. In deep learning, it helps train neural networks by updating weights so the predicted becomes closer to the actual outputs.

② What is the role of the learning rate in Gradient Descent?

→ The learning rate controls the size of the step taken when updating model's parameters during Gradient Descent. A small learning rate leads to slow convergence, while a very large learning rate can cause the algorithm to overshoot the minimum and fail to converge.

③ Define batch Gradient Descent.

→ Batch Gradient Descent is a type of Gradient Descent where the gradient of the loss function is calculated using the entire training datasets before updating the model parameters in each iteration.

④ What problem of Gradient Descent is addressed by Momentum-based Gradient Descent?

→ Momentum-based Gradient Descent addresses the problem of slow convergence and oscillations in standard Gradient Descent, especially in areas with steep and narrow curves. It accelerates learning by considering the previous update direction while updating parameters.

⑤ What is Nesterov Accelerated Gradient (NAG)?

→ Nesterov Accelerated Gradient is an improved version of momentum-based Gradient Descent where the gradient is calculated after making a temporary step in the direction of the previous momentum, allowing more accurate and faster convergence.

⑥ How does Stochastic Gradient Descent differ from batch Gradient Descent?

→ Stochastic Gradient Descent (SGD) updates the model parameters using only one training example at a time, while batch Gradient Descent uses the entire training dataset to compute the gradient before each update.

⑦ What are the advantages of using SGD for large datasets?

→ SGD is computationally faster & requires less memory because it processes one sample at a time. It also helps models escape local minima and works efficiently for very large datasets.

⑧ What is Principal Component Analysis (PCA)?

→ Principal Component Analysis is a statistical technique used to reduce the dimensionality of a dataset while preserving as much variance (information) as possible by transforming the data into a new set of orthogonal variables called principal components.

⑨ What is meant by dimensionality reduction?

→ Dimensionality reduction is the process of reducing the number of input variables (features) in a dataset while retaining the most important information.

⑩ What is the geometric interpretation of PCA?

→ Geometrically, PCA finds new orthogonal axes (Principal components) that maximize the variance of the projected data. The first Principal component captures the direction of maximum variance, and the following components capture the remaining variance while being perpendicular to the previous ones.

⑪ Define Singular Value Decomposition (SVD).

→ Singular Value Decomposition is a matrix factorization technique that decomposes a matrix A into three matrices U , Σ , and V^T , such that $A = U\Sigma V^T$. It is widely used in data analysis, signal processing, and machine learning.

(12) How is SVD related to PCA?

→ SVD is used to compute PCA. When SVD is applied to a centered data matrix, the principal components correspond to the right singular vectors, and the singular values related to the variance explained by each component.

(13) What are model parameters in deep learning?

→ Model parameters are the internal variables of a neural network that are learned from training data, such as weights and biases.

(14) What are hyper-parameters?

→ Hyper-parameters are configuration settings that are defined before training the model and control how the training process happens.

(15) Give two examples of each parameters & hyper-parameters.

→ Examples of parameters: weights and biases.
examples of hyperparameters: learning rate & batch size.

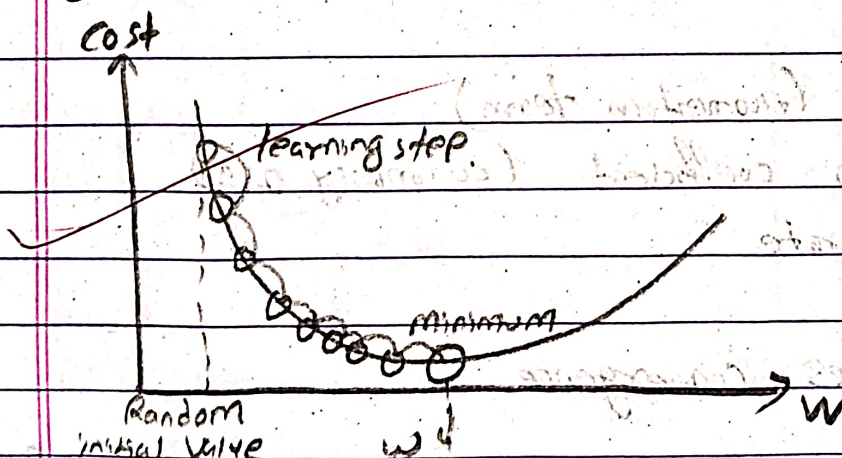
Long Questions

① Explain the Gradient Descent algorithm in detail. Discuss its working & limitations.

→ GD is an optimization algo used to minimize the loss (cost) function of a ML or DL model by iteratively updating model parameters such as weights & biases.

Working of GD:

- I) Initialize parameters (weights & biases) with random values.
- II) Compute the loss function using the current parameters.
- III) Calculate the gradient, which represents the slope of the loss function with respect to each parameter.
- IV) Update the parameters in the opposite direction of the gradient to reduce the error and repeat process.



$$\theta = \theta - \eta \nabla J(\theta)$$

where,

θ = model parameters

η = learning rate

$\nabla J(\theta)$ = gradient of the loss function

Limitations:

- may get stuck in local minima or saddle points
- slow convergence in deep networks
- computationally expensive for large dataset when using small batches.

② Describe momentum based Gradient Descent and explain how it improves convergence speed.

→ Momentum Based Gradient Descent improves the standard Gradient Descent algorithm by adding a velocity term that accumulates past gradients. This helps the optimization move faster in consistent directions.

Working

Instead of only using the current gradient, the update also considers the previous step.

$$V_t = \beta V_{t-1} + \eta \nabla J(\theta)$$

$$\theta = \theta - V_t$$

where?

V_t = velocity (momentum term)

β = momentum coefficient (commonly 0.9)

η = learning rate

How it improves convergence.

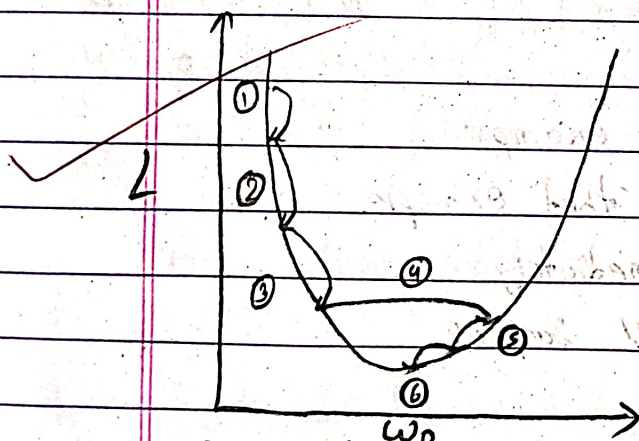
- Reduces oscillation in steep regions of the loss ~~landscape~~ ^{surface}.
- Helps reach the global minimum faster.
- Accelerates movement in the correct direction.

③ Explain Nesterov Accelerated Gradient descent and compare it with momentum GD

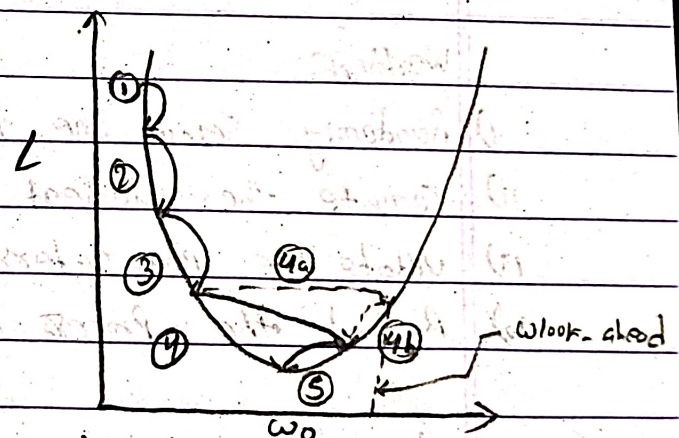
→ Nesterov Accelerated Gradient (NAG) is an improved of momentum GD. Instead of computing the gradient at the current parameters, NAG first moves in the direction of the previous momentum and then calculates the gradient.

Steps

- i) Predict the next position: $\theta' = \theta - \beta v_{t-1}$
- ii) compute gradient at this look-ahead point
- iii) Update velocity and parameters.



a) Momentum - GD



b) Nesterov Accelerated GD

Comparison

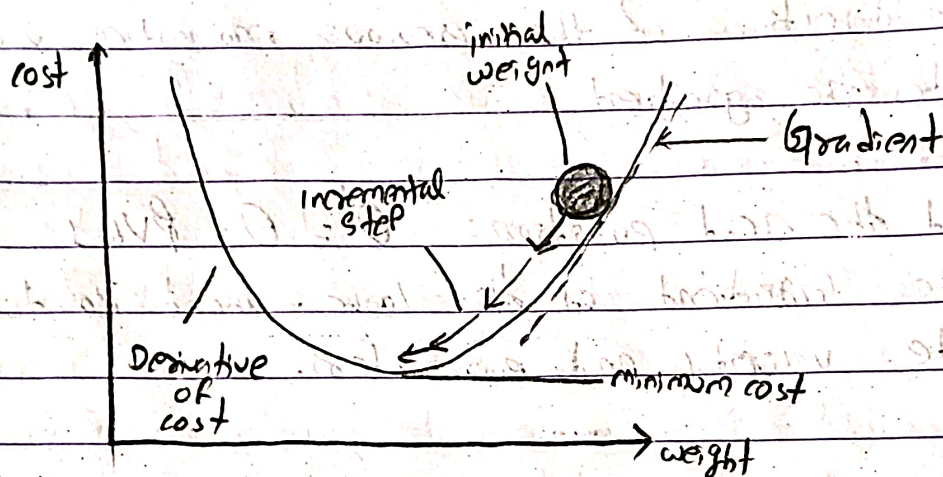
Momentum GD

Nesterov GD

- | | |
|--|---|
| i) Gradient calculation at current position. | i) calculated at predicted future position |
| ii) Good accuracy | ii) more precise |
| iii) Faster convergence than basic GD | iii) usually faster convergence and more stable |

Q) Explain Stochastic Gradient descent (SGD) with its advantages & disadvantages.

→ Stochastic Gradient descent updates model parameters using one training sample at a time instead of using the whole dataset.



Working:

- i) Randomly select one training example
- ii) Compute the gradient for that example.
- iii) Update the parameters immediately.
- iv) Repeat the process for all samples

Advantages

- Faster updates compared to batch gradient descent.
- Requires less memory
- Works well with very large datasets

Disadvantages:

- Updates are noisy & unstable.
- Requires careful tuning of learning rate.

⑤ Explain the working principle of PCA along with its mathematical intuition.

→ Principal Component Analysis (PCA) is a technique for to reduce the number of features in a dataset while retaining most of the important information.

Working Principle:

- i) Standardize the dataset
- ii) Compute the covariance matrix of the data.
- iii) Calculate eigenvalues and eigenvectors of the covariance matrix.
- iv) Select the top ~~eigenvalues~~ eigenvectors with the largest eigenvalues.
- v) Transform the original data into the new coordinate system.

Mathematical intuition:

- Eigenvectors represent principal directions of the data.
- Eigenvalues represent the amount of variance captured by each direction.
- PCA projects data ~~into~~ onto directions where variance is maximum.

⑥ Discuss different interpretations of PCA, including variance maximization and decorrelation.

→ Variance maximization:

- PCA finds directions (Principal components) where the data variance is maximum. The first component captures the highest variance, the second captures the next highest variance while being orthogonal to the first.

Decorrelation:

- PCA transforms correlated variables into uncorrelated variables (principal components). The covariance between principal components becomes zero.

Dimensionality Reduction:

- By selecting only the top principal components, PCA reduces the number of features while preserving most of the information.

⑦ Explain Singular Value Decomposition (SVD) and its role in machine learning.

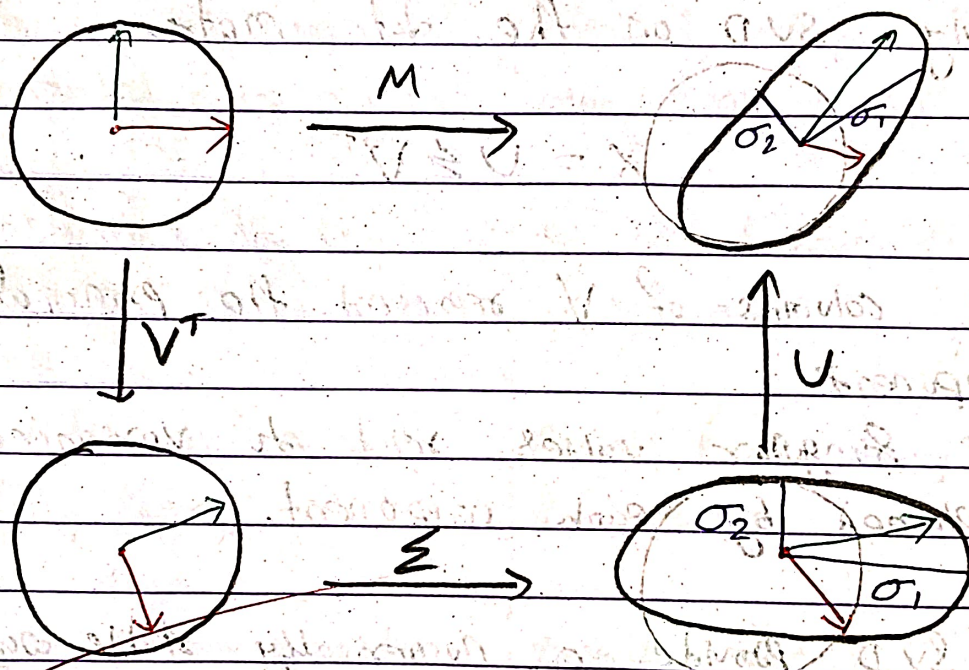
→ Singular value decomposition is a matrix factorization technique used in linear algebra and machine learning.

It decomposes a matrix A , into three matrices: $A = U \Sigma V^T$

where,

- U = left singular vectors
- Σ = diagonal matrix of singular values
- V^T = right singular vectors.

Example:



$$M = U \cdot \Sigma \cdot V^T$$

Role in Machine Learning

- Dimensionality reduction
- Noise reduction
- Data compression
- Recommendation systems
- Natural language processing.

⑧ How are PCA and SVD related? Explain with a clear explanation.

→ PCA can be computed using SVD

Steps:

- 1) First center the dataset by subtracting the mean
- 2) Apply SVD on the data matrix

$$X = U \Sigma V^T$$

- 3) The columns of V represent the principal components
- 4) The singular values relate to variance explained by each component.

So, SVD provides a numerically stable way to compute PCA.

⑨ Explain the difference between parameters and hyperparameters in deep learning models.

→ The difference between parameters and hyperparameters are as:-

Parameters	Hyper-parameters
i) Internal values learned by the model during training	External settings defined before training starts
ii) It determine how inputs are transformed into outputs	It control how the model learns
iii) Updated continuously during training	Remain fixed during a training run
iv) Learned using algorithms like Gradient descent and Backpropagation	Tuned using techniques like Grid search or Random Search
v) Examples: weights, biases	Examples: Learning rate, batch size, number of epochs, number of layers

⑩ Discuss how the choice of hyper-parameters affects model performance.

→ Hyper-parameters strongly influence the training process and final performance.

Learning Rate:

- Too high → training becomes unstable
- Too low → very slow learning

Batch size

- small batch → noisy but better generalization
- large batch → stable but requires more memory

Number of Epochs

- too few → underfitting
- too many → overfitting

Model Architecture

- more layers or neurons increase model capacity but may cause overfitting.

Proper tuning of hyper-parameters helps achieve better accuracy, faster convergence, and improved generalization.

